

UART Transmitter Design using VHDL Vivado

Aim: To design and simulate UART transmitter using VHDL in Vivado.

Introduction:

UART (Universal Asynchronous Receiver Transmitter) is a serial communication protocol used for transmitting and receiving data between digital devices. It is widely used in embedded systems, microcontrollers, FPGA boards, computers, and communication modules.

UART is used because it provides a simple and reliable method of serial data transfer without requiring a separate clock signal. It reduces wiring complexity by sending data one bit at a time through a transmit line (TX) and receiving through a receive line (RX).

Common applications of UART include communication between FPGA and computer, microcontroller to sensors, Bluetooth modules, GPS modules, and IoT devices.

VHDL CODE:

```
library IEEE;  
use IEEE.STD_LOGIC_1164.ALL;  
use IEEE.NUMERIC_STD.ALL;
```

entity uart_tx is

```
Port ( clk : in STD_LOGIC;  
      reset : in STD_LOGIC;  
      tx_start : in STD_LOGIC;  
      data_in : in STD_LOGIC_VECTOR (7 downto 0);  
      tx : out STD_LOGIC;  
      tx_busy : out STD_LOGIC);
```

end uart_tx;

architecture Behavioral of uart_tx is

```
signal bit_count : integer range 0 to 9 := 0;  
signal shift_reg : STD_LOGIC_VECTOR(9 downto 0);  
signal busy : STD_LOGIC := '0';
```

begin

```
tx_busy <= busy;  
tx <= shift_reg(0);
```

process(clk, reset)

begin

```
if reset = '1' then  
    bit_count <= 0;  
    shift_reg <= (others => '1');  
    busy <= '0';
```

```
elsif rising_edge(clk) then
```

```
if tx_start = '1' and busy = '0' then  
    shift_reg <= '1' & data_in & '0';
```

```

        bit_count <= 0;
        busy <= '1';

    elsif busy = '1' then
        shift_reg <= '1' & shift_reg(9 downto 1);

        if bit_count = 9 then
            busy <= '0';
        else
            bit_count <= bit_count + 1;
        end if;

    end if;

end if;

end process;

end Behavioral;

```

Testbench code:

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity uart_tx_tb is
end uart_tx_tb;

```

architecture Behavioral of uart_tx_tb is

component uart_tx

Port (

clk : in STD_LOGIC;

reset : in STD_LOGIC;

tx_start: in STD_LOGIC;

data_in : in STD_LOGIC_VECTOR(7 downto 0);

tx : out STD_LOGIC

);

end component;

signal clk : STD_LOGIC := '0';

signal reset : STD_LOGIC := '1';

signal tx_start : STD_LOGIC := '0';

signal data_in : STD_LOGIC_VECTOR(7 downto 0) := "10101010";

signal tx : STD_LOGIC;

begin

uut: uart_tx

port map (

clk => clk,

reset => reset,

tx_start => tx_start,

data_in => data_in,

tx => tx

);

```
clk_process : process
```

```
begin
```

```
    while true loop
```

```
        clk <= '0';
```

```
        wait for 10 ns;
```

```
        clk<='1';
```

```
        wait for 10 ns;
```

```
    end loop;
```

```
end process;
```

```
stimulus: process
```

```
begin
```

```
    reset<='1';
```

```
    wait for 20ns;
```

```
    reset<='0';
```

```
    wait for 20 ns;
```

```
    tx_start<='1';
```

```
    wait for 20ns;
```

```
    tx_start<='0';
```

```
    wait;
```

```
end process;
```

```
end Behavioral;
```

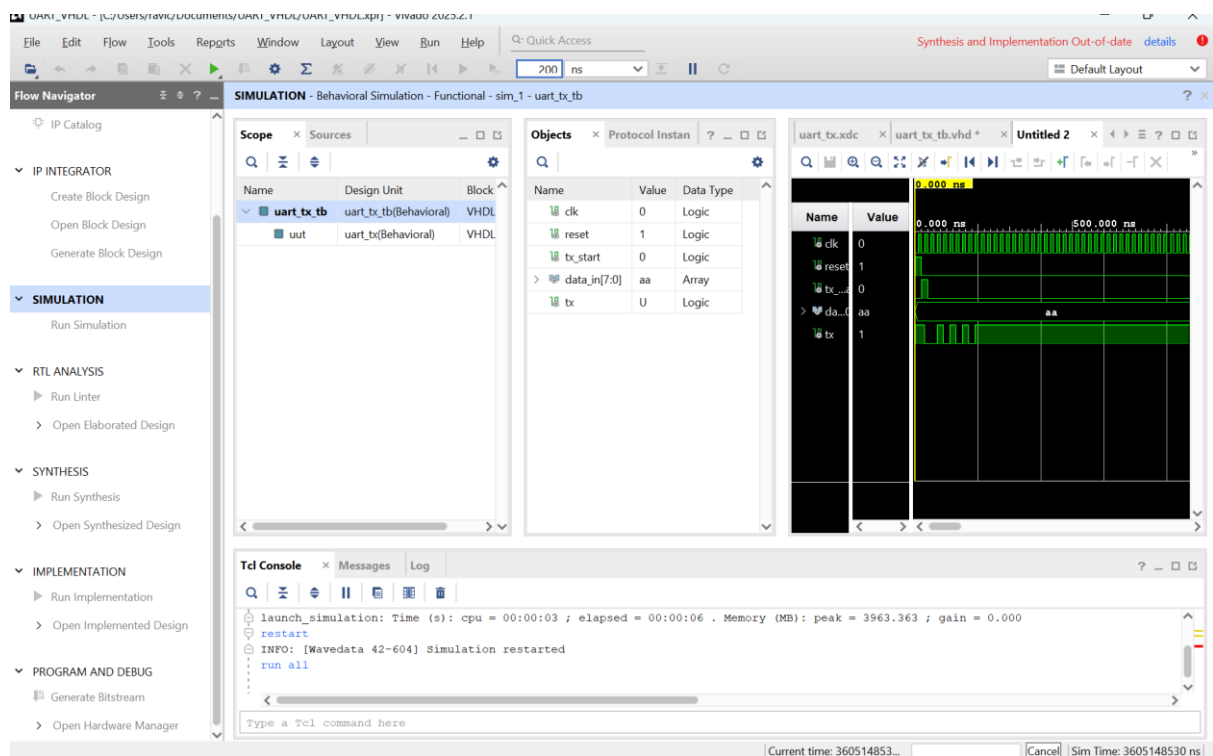
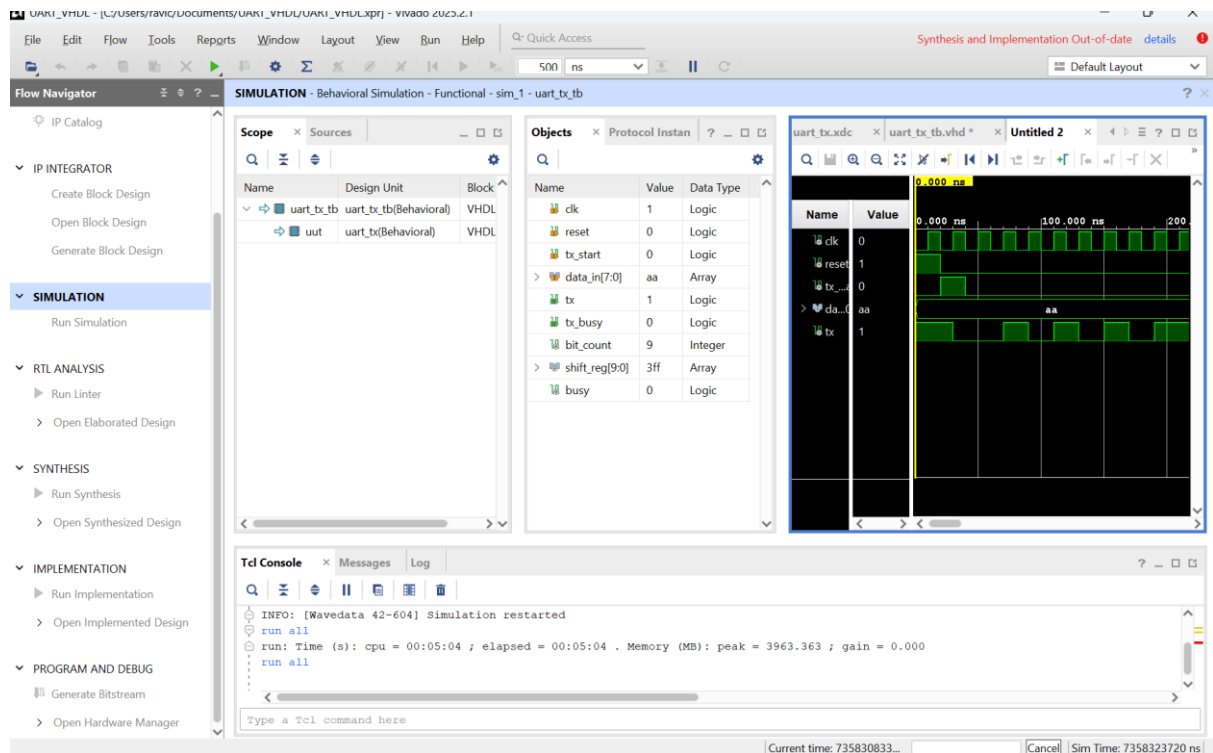
Working:

In UART communication, data is transmitted serially in asynchronous form. A start bit is sent first, followed by the data bits, and then a stop bit.

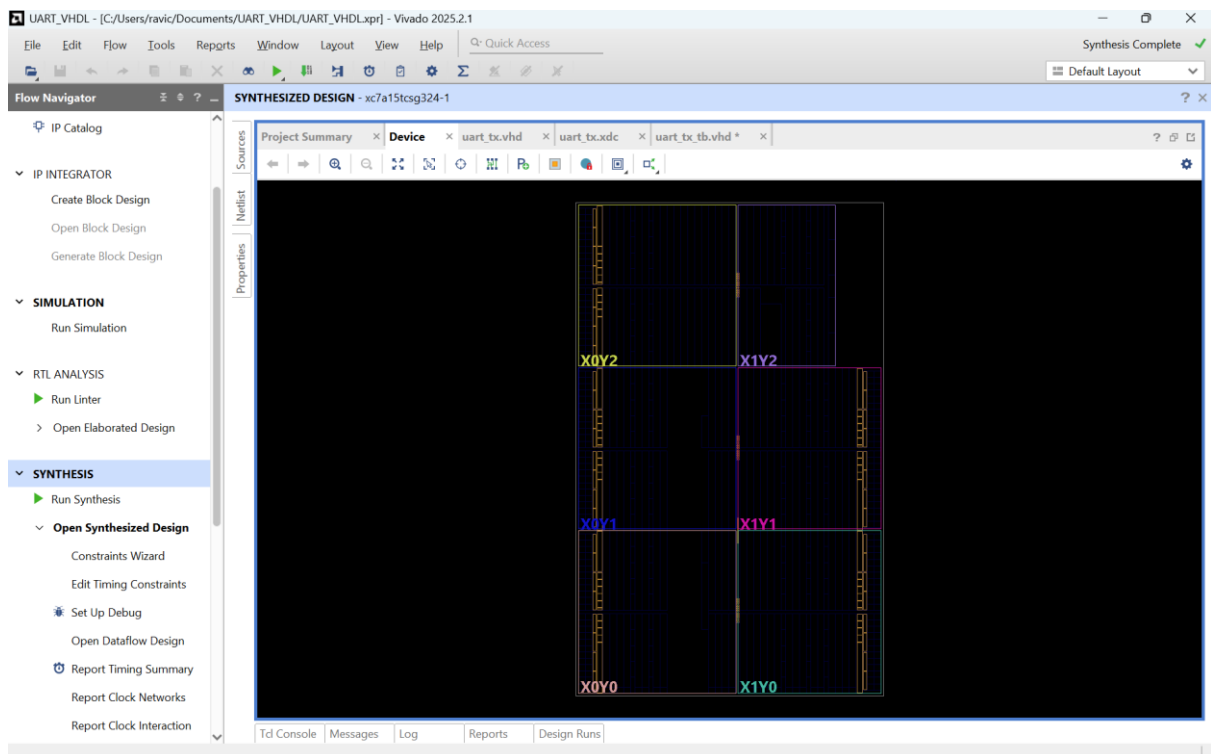
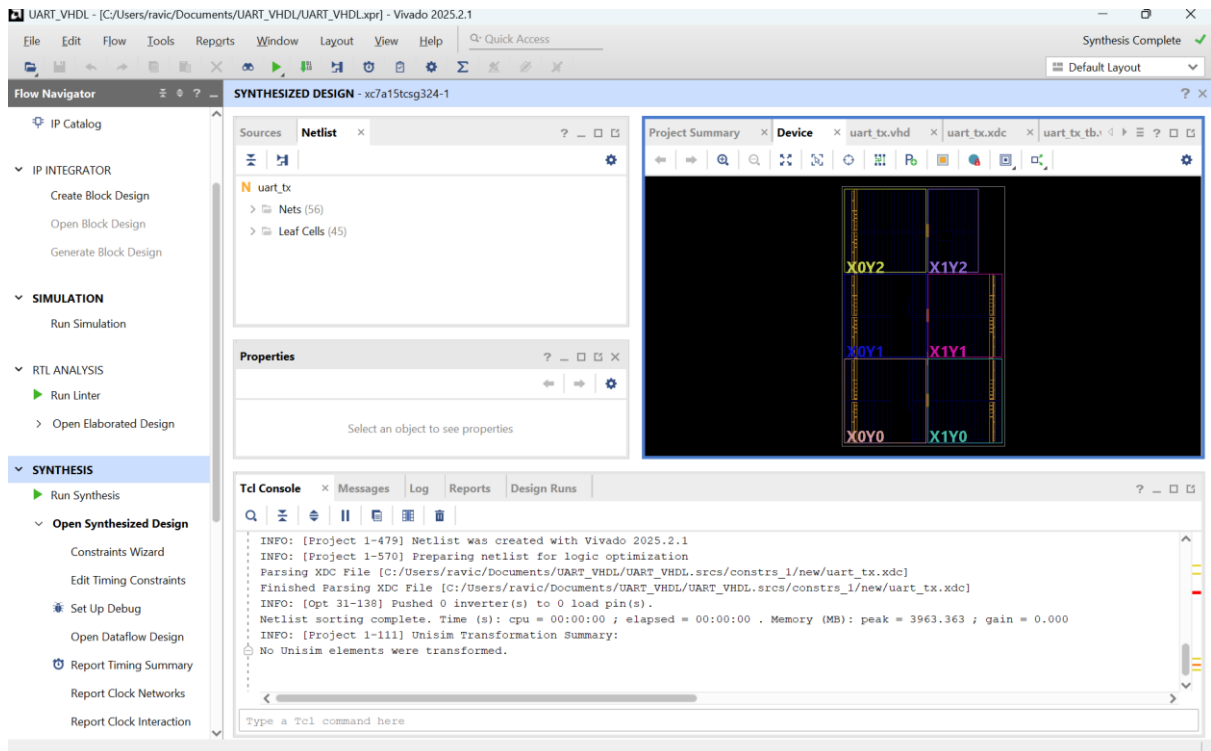
When transmission starts, the transmitter converts parallel input data into serial output and sends each bit through the TX line. The receiver reads the bits in sequence and reconstructs the original data.

Since UART does not use a clock signal, both transmitter and receiver operate using the same baud rate to maintain synchronization.

Simulation output screenshot



Synthesis screenshot



UART_VHDL - [C:/Users/ravic/Documents/UART_VHDL/UART_VHDLxpr] - Vivado 2025.2.1

File Edit Flow Tools Reports Window Layout View Help Quick Access

Synthesis Complete ✓

Default Layout

Flow Navigator Paste (Ctrl+V) HESIZED DESIGN - xc7a15tcsq324-1

IP Catalog

IP INTEGRATOR

- Create Block Design
- Open Block Design
- Generate Block Design

SIMULATION

- Run Simulation

RTL ANALYSIS

- Run Linter
- Open Elaborated Design

SYNTHESIS

- Run Synthesis
- Open Synthesized Design
 - Constraints Wizard
 - Edit Timing Constraints
 - Set Up Debug
 - Open Dataflow Design
 - Report Timing Summary
 - Report Clock Networks
 - Report Clock Interaction

Project Summary x Device x uart_tx.vhd x uart_tx.xdc x uart_tx_tb.vhd x

Overview Dashboard

Settings Edit

Project name: UART_VHDL

Project location: C:/Users/ravic/Documents/UART_VHDL

Product family: Artix-7

Project part: xc7a15tcsq324-1

Top module name: uart_tx

Target language: VHDL

Simulator language: Mixed

Target Simulator: Vivado Simulator

Synthesis

Status: ✓ Complete

Messages: No errors or warnings

Part: xc7a15tcsq324-1

Strategy: Vivado Synthesis Defaults

Report Strategy: Vivado Synthesis Default Reports

Incremental synthesis: Automatically selected checkpoint

Implementation

Status: Not started

Messages: No errors or warnings

Part: xc7a15tcsq324-1

Strategy: Vivado Implementation Defaults

Report Strategy: Vivado Implementation Default Reports

Incremental implementation: None

DRC Violations

Run Implementation to see DRC results

Timing

Run Implementation to see timing results

Td Console Messages Log Reports Design Runs

Paste the clipboard contents

UART_VHDL - [C:/Users/ravic/Documents/UART_VHDL/UART_VHDLxpr] - Vivado 2025.2.1

File Edit Flow Tools Reports Window Layout View Help Quick Access

Synthesis Complete ✓

Default Layout

Flow Navigator SYNTHESIZED DESIGN - xc7a15tcsq324-1

IP Catalog

IP INTEGRATOR

- Create Block Design
- Open Block Design
- Generate Block Design

SIMULATION

- Run Simulation

RTL ANALYSIS

- Run Linter
- Open Elaborated Design

SYNTHESIS

- Run Synthesis
- Open Synthesized Design
 - Constraints Wizard
 - Edit Timing Constraints
 - Set Up Debug
 - Open Dataflow Design
 - Report Timing Summary
 - Report Clock Networks
 - Report Clock Interaction

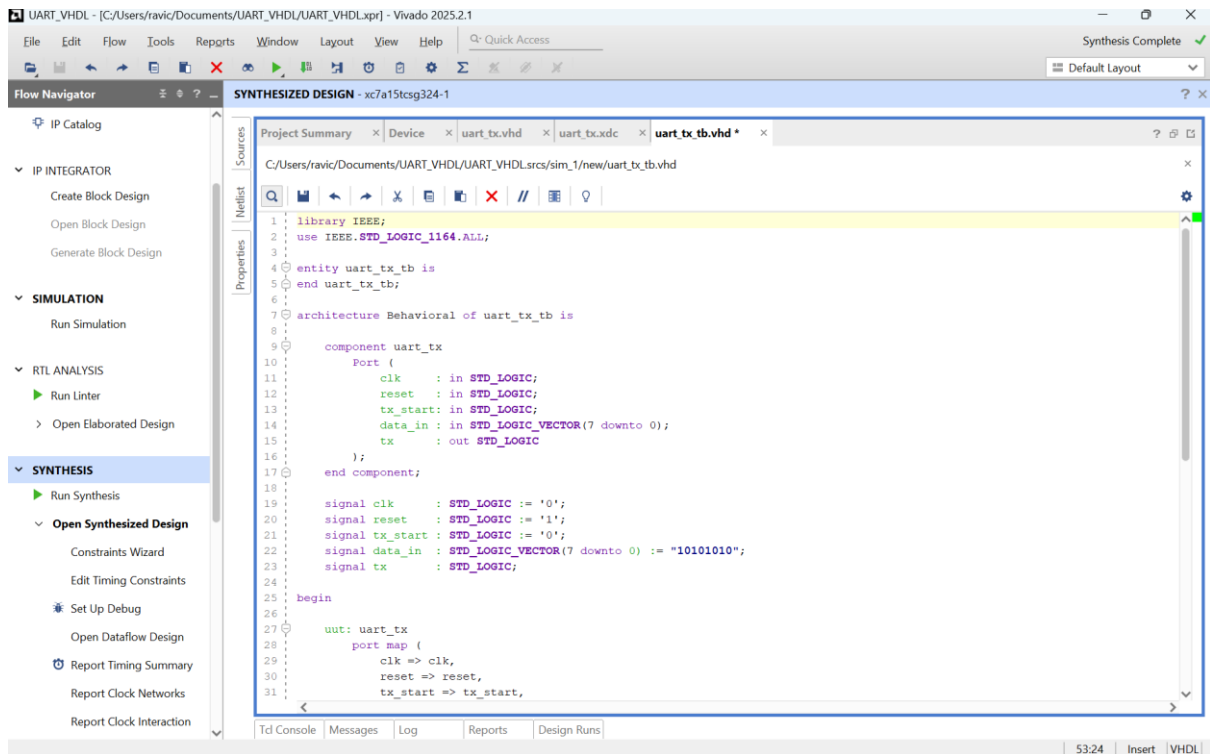
Project Summary x Device x uart_tx.vhd x uart_tx.xdc x uart_tx_tb.vhd x

C:/Users/ravic/Documents/UART_VHDL/UART_VHDLsrcs/sources_1/new/uart_tx.vhd

```
35 entity uart_tx is
36   Port ( clk : in STD_LOGIC;
37         reset : in STD_LOGIC;
38         tx_start : in STD_LOGIC;
39         data_in : in STD_LOGIC_VECTOR (7 downto 0);
40         tx : out STD_LOGIC;
41         tx_busy : out STD_LOGIC);
42 end uart_tx;
43
44 architecture Behavioral of uart_tx is
45   signal bit_count : integer range 0 to 9 := 0;
46   signal shift_reg : STD_LOGIC_VECTOR(9 downto 0);
47   signal busy : STD_LOGIC := '0';
48
49 begin
50   tx_busy <= busy;
51   tx <= shift_reg(0);
52
53   process(clk, reset)
54   begin
55     if reset = '1' then
56       bit_count <= 0;
57       shift_reg <= (others => '1');
58       busy <= '0';
59
60     elsif rising_edge(clk) then
61
62       if tx_start = '1' and busy = '0' then
63         shift_reg <= '1' & data_in & '0';
64         bit_count <= 0;
65         busy <= '1';
```

Td Console Messages Log Reports Design Runs

47:31 Insert VHDL



Advantages:

- Simple communication method
- Requires only TX and RX lines
- Low hardware complexity
- Reliable serial communication
- Useful in FPGA and embedded applications

Conclusion:

The UART Transmitter was successfully designed using VHDL in Vivado. The design was verified through behavioural simulation, and the output waveform confirmed correct serial transmission of the input data. The synthesized design was generated successfully without functional errors. Hence, the UART transmitter implementation was completed successfully.

Result:

The UART Transmitter was implemented using VHDL and verified in Vivado. The simulation waveform showed correct transmission of data, and synthesis completed successfully. Therefore, the design achieved the expected UART serial communication operation.